Extension Everything Under the Covers @ILSpy

Siegfried, Christoph

Who are these guys?

- Siegfried Pammer
  - https://github.com/siegfriedpammer/
- Christoph Wille
  - https://github.com/christophwille

Where to Find Us

ILSpy Team
@ilspy@hachyderm.io

The ILSpy Team is now also on Mastodon! Been mulling a couple of options, but the clear guidelines at github.com/hachyderm/community... won us over.

GitHub
community/open-source-accounts.md at main...
Hachyderm Community Resources. Contribute to hachyderm/co...

Dec 20, 2022, 10:14 AM · · Web

ILSpy - .NET Decompiler
@ilspy.bsky.social

You might be wondering what is the difference between our bsky and hachyderm.io/@ilspy accounts? The latter is "channel 9 from the cockpit" (or chatter from the engine room) with more details about the development of ILSpy vs more informational/support style of posts on bsky.

7:45 AM · Nov 11, 2024 Everybody can reply

“Traditional” Talk Structure (aka Agenda)

- Engineering challenges & changes
- Past predictions & future predictions
- One language feature under the covers
  - This time: Extension Everything

2022 Edition: AMA - Ask Me Anything with the ILSpy Team

https://youtu.be/rexge_m4JR0?t=1186

A recap from last time July ‘22

What worked out, what didn’t age well, what fell outright on its face

ILSpy Team @ilspy · Apr 17

Coming soon.... 😀#ILSpy #csharp

```
erable<object> @0600001a
Decompiler.Tests.TestCases.Pretty.Yie
llections.Generic;

ble<object> YieldBangBang(object x!

n x;
```

5 6 59

Commits on Jul 23, 2024

Revert "Add support for C# 11 parameter null checks"

dgrunwald committed on Jul 23, 2024

Commits on Apr 19, 2022

Disable DecompilerSettings.ParameterNullCheck

siegfriedpammer committed on Apr 19, 2022

Commits on Apr 16, 2022

Add support for C# 11 parameter null checks

siegfriedpammer committed on Apr 16, 2022

Last time…

- Our journey from net472 to net60
- How and Why TestRunner
- ILSpyX for non-WPF frontends
- Supply chain security

Releases Since (including oopsies)

- 8.0, 8.1 (+8.1.1), 8.2, 9.0, 9.1, 10.0 Previews 1 & 2

Split out VS Extensions into separate sln & add arm64 binaries to 2022 VSIX #3009

Edit | Code

Merged

Conversa

chri

See

Cannot open a DLL produced with .NET 10 Alpha using ILSpy 8.2.0.7535 #3349

Closed

carlossanlop opened on Dec 13, 2024

I created a simple console app that targets .NET 10 alpha (latest available nightly) It compiled without issues. (I shared the csproj and cs code at the end for a repro).

If I try to open the produced assembly, I get an error using this ILSpy version:

OpenSSF Scorecard

ILSpy Team
@ilspy@hachyderm.io

You might have seen a few commits recently that had nothing to do with #ILSpy per se - but with improving our OpenSSF score card (least privilege for workflow tokens, security.md, actually putting LICENSE where it always was supposed to be...) - and it paid off! We went up to seven (from a not all-too-shabby 5.3).

https://securityscorecards.dev/viewer/?uri=github.com%2Ficsharpcode%2FILSpy

- [ ] Try to see if Sonar finds anything interesting ✓
  #3366 by christophwille was closed on Jun 24 • Draft
- [ ] Update Scorecard actions versions and apply Zizmor offline findings ✓ Build Automation github_actions
  #3365 by christophwille was merged on Jan 11
- [ ] Create scorecard.yml ✓ Build Automation github_actions
  #3362 by christophwille was merged on Jan 7
- [ ] Principle of least privilege for workflow tokens ✓ Build Automation
  #3360 by christophwille was merged on Jan 3
- [ ] Enable repeatable package restores ✓ Build Automation
  #3359 by christophwille was merged on Jan 3 • Approved
- [ ] Move license.txt to root folder LICENSE file ✓ Documentation
  #3358 by christophwille was merged on Jan 3

Microsoft.Sbom.Targets

Use Microsoft.Sbom.Targets in ICSharpCode.Decompiler NuGet

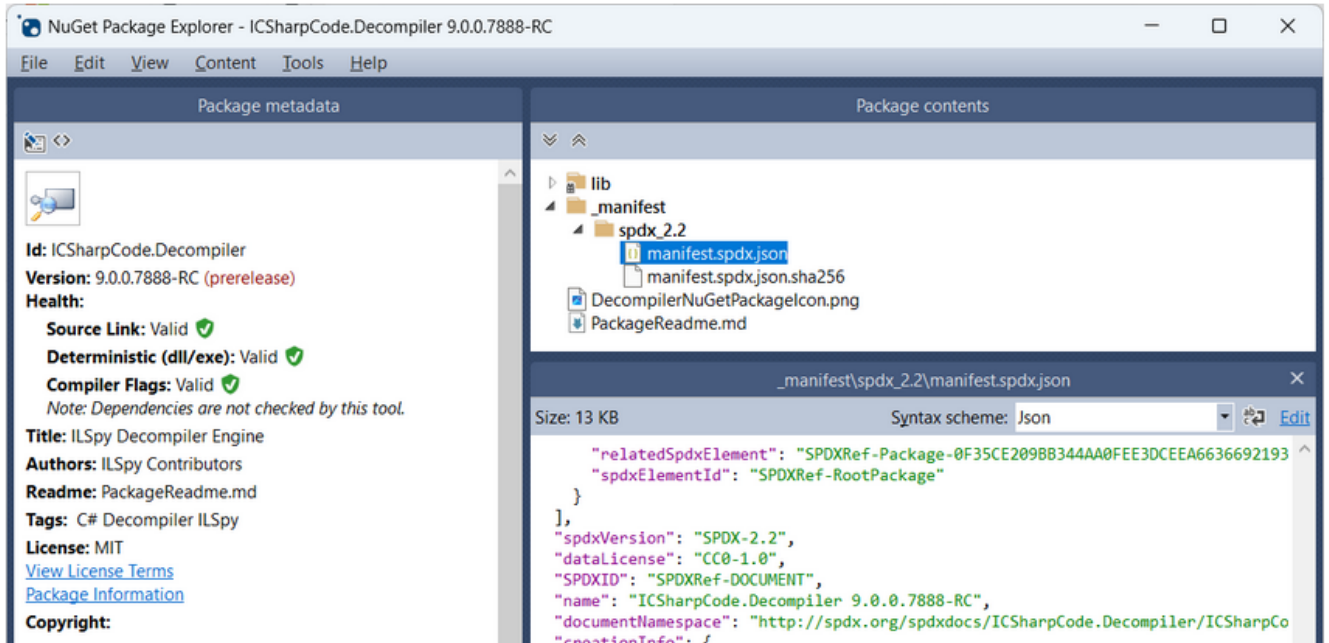
Merged **christophwille** merged 10 commits into `master` from `christophwille/sbomtargets` on Jan 10, 2025

Conversation 0 | Commits 10 | Checks 9 | Files changed 8

christophwille commented on Dec 4, 2024 • edited — Member

https://github.com/microsoft/sbom-tool/tree/main/src/Microsoft.Sbom.Targets to embed SBOM

- GeneratePackageOnBuild removed, build action now has a new step for dotnet pack
- Decompiler, BamlDecompiler and ILSpyX include SBOM



OSS “Drama”

Switch back to FluentAssertions 7.0 #3370

Merged christophwille merged 2 commits into `master` from `fix/revertfluentassertions` on Jan 15, 2025

Switch from FA to Shouldly #3371

Merged siegfriedpammer merged 1 commit into `master` from `fix/fabegone` on Jan 16, 2025

Conversation 0 | Commits 1 | Checks 9 | Files changed 5

christophwille commented on Jan 16, 2025 Member

No description provided.

Switch from FA to Shouldly

ILSpy Team
@ilspy
Jan 15, 2025

The background for this github.com/icsharpcode/ILSpy/p... - if a commercial entity uses the NuGet, FA 8 would not be a problem. However, if they fork (remember, we are MIT), they'd be in license trouble with our tests - most likely not even knowing about it! We want anyone to be able to use our entire codebase for whatever purposes without hidden surprises.

WPF Refactoring

- ILSpy started in 2011
- MVVM? We've heard of it
- v9.0 saw a bunch of refactorings

> Various WPF-related refactorings by **@tom-englert** (see #3257, #3266, #3274, #3283, #3285, #3291, #3292, #3294, #3295, #3297, #3298, #3299, #3302, #3308, #3314, #3325 and #3335)

- and now talk about "self-inflicted pain"

Microsoft Testing Platform

- A general recommendation for everyone to look into

Move from dotnet.config to global.json #3553

Merged christophwille merged 3 commits into icsharpcode:master from Youssef1313:patch-1 on Oct 15, 2025

Conversation 3 | Commits 3 | Checks 4 | Files changed 2

All commits

Filter files...

- dotnet.config
- global.json

dotnet.config

global.json

```
@@ -3,5 +3,8 @@
3  3      "version": "10.0.0",
4  4      "rollForward": "major",
5  5      "allowPrerelease": true
   6  +  },
   7  +  "test": {
   8  +    "runner": "Microsoft.Testing.Platform"
6  9    }
7     - }
```

CompareOptions.NumericOrdering

```
- using NaturalSort.Extension;
+ using System.Globalization;

  namespace ICSharpCode.ILSpy.TreeNodes
  {
  @@ -28,6 +27,6 @@ namespace ICSharpCode.ILSpy.TreeNodes
      /// </summary>
      public sealed class NaturalStringComparer
      {
-         public static readonly IComparer<string> Instance = StringComparison.CurrentCultureIgnoreCase.WithNaturalSort();
+         public static readonly IComparer<string> Instance = StringComparer.Create(CultureInfo.CurrentCulture, CompareOptions.NumericOrdering);
      }
  }
```

Input: A, A10, A11, Z, A2, A22

NaturalSort.Extensions: A, A2, A22, A10, A11, Z

StrCmpLogicalW: A, A2, A10, A11, A22, Z

Translations

- Removed

christophwille commented last week • edited

Member

Thank you for shining a bright light on an area of ILSpy we haven't had on our radar for years, and in fact dreaded to address. For one, there is the technical issue that adding another language is a hassle, as modifying csprojs should never be needed for making changes to supported languages.

Secondly, there are additional "costs" of having translations that bothered us for a long time:

- translation drift because of renames (menu items, options, error messages... you-name-it)
- terminology in translations / bug reports about untranslated strings
- no way for us (core team) to validate the tran
- no way for us (core team) to maintain translat
  Copilot #3551)
- given how highly technical the audience is, a

Therefore, sorry, we cannot merge this PR and ac

A future version of translation must have the following properties:

- Single-file user-droppable resource file
- Picked up by ILSpy after restart automatically
- No more magic inside any of the project files

This approach allows third-parties to ship translations without our help/integration inside deliverables.

There are no plans nor timeline for this future version because we think English should be good enough for a highly technical audience.

Smaller Fish

Make Tests Great Again #3231

Merg

Enable `<RollForward>major</RollForward>` for `ILSpy.exe` #3391

Cor

Merged siegfriedpammer merged 1 commit into `icsharpcode:master` from `nil4:master` on Feb 14, 2025

Conversation 1 | Commits 1 | Checks 5 | Files changed 1

Add IFileLoader API #3191

Merged siegfriedpammer merged 3 commits into `master` from `file-loaders` on Apr 1, 2024

Conversation 1 | Commits 3 | Checks 0 | Files changed 20

siegfriedpammer commented on Mar 31, 2024 Member

No description provided.

iers>

<AutoGenerateBindingRedirects>false</AutoGenerateBindingRedirects>

Avalonia(ILSpy)

Jul 19, 2024

ILSpy Pu

ILSpy Team
@ilspy@hachyderm.io

Jun

2025 2

12-20

December 20, LeXtudio published the first public preview of Project Rover, a new effort to bring ILSpy to macOS and Linux.

company:lextudio | product:ilspy | product:avalonia

07-21

July 21, the AvaloniaILSpy GitHub repository was archived. This was started by Jeffrey Ye in August 2018.

product:ilspy | product:avalonia

Slopzilla, aka AI

Close encounters of the 3rd kind

- Explain this code (Jan 23)
- Release notes (9.0P1, Ap
- Translations (Sep 25)
- Yes, AI-generated PRs dic
- Copilot reviews are trigge
- MCP/Tool calling discuss
  - https://trustedsec.com/blog/h
  - https://github.com/diskodan/C
- Analysis of decompilatio

RestSharp.113.0.0.0.md
Markdown

This file contains a diff summary of a C# project decompiled from a binary. Can you point out all locations where there is still compiler-generated code or other problems? The goal is to replace all code that is compiler-generated with the original constructs and have a project that compiles.

Overview

You've got a mix of decompiler artifacts, compiler-generated stubs, and project-level issues that will prevent a clean compile or produce brittle behavior. Below is a file-by-file, actionable audit of all the remaining compiler-generated constructs, stubs, and other problems, plus concrete fix guidance.

Critical compiler-generated artifacts and stubs

These are synthetic constructs emitted by the compiler or decompiler. They must be removed or

Dev Days 2024 Surprise

Start work on upgrading Roslyn 4.9 to 4.10.0 (upgraded dotnet format #3238 for proper formatting)

Saturday

Roslyn 4.10 continued lifted operators, compound assignments (detour ILInlining)

Sunday

- Compound assignments string.Concat, thus upgrading Roslyn 4.10 completed

Moving Too Early, Revisited

Test Roslyn 5 from dotnet-tools transport feed #3517

Closed

Availability of Microsoft.Net.Compilers.Toolset on NuGet matching VS2026 Insiders #80213

Convers

Closed Task

Roslyn 5.0.0 #3588

Merged christophwille merged 3 commits into master from r502 on Nov 30, 2025

Contributor

slyn/blob/main/docs/

Conversation 10 Commits 3 Checks 9 Files changed 15

christophwille commented on Oct 10, 2025 • edited

Member

dotnet/roslyn#80213

#3539

The Future

- ASTalavista, baby!
- Focus on language features

- Will we move too fast again?
  - Unions
  - Runtime async

```
extension<T>(IEnumerable<T> @this)
{
    public bool IsEmpty => !@this.Any();
}

extension(string @this)
{
    public int WordCount
        => Regex.Matches(@this, @"\w+").Count;
}
```

Extensions

Extension blocks

- New **`extension`** block syntax usable in **`static`** class „containers“
- Moves **`this`** (“receiver”) parameter “out” of the member declaration
- Supports generic parameters and constraints
- Fully backwards compatible
- All members are accessible via method call syntax

Receiver parameters

- Receiver parameter can use any parameter modifier valid with “classic” `this` parameter (think ref, in)
- Receiver parameter name optional, if you only need static extensions

```
extension<T>(IEnumerable<T> source)
{
    public bool IsEmpty => !source.Any();

    public static IEnumerable<T> Empty => Enumerable.Empty<T>();

    public static IEnumerable<T> operator +(
        IEnumerable<T> left,
        IEnumerable<T> right
    ) => left.Concat(right);

    public T FirstOrDefault(T v)
    {
        return source.FirstOrDefault() ?? v;
    }
}
```

Quiz

```
Test("Hello, World!");

static void Test(string text)
{
    Console.WriteLine("Test: " + text);
}

public static class Extensions
{
    extension(string)
    {
        public static string operator +(string left, string right)
        {
            return "Hijacked!";
        }
    }
}
```

What does this code do?

A) Compiler error or warning

B) Print “Test: Hello, World!”

C) Print “Hijacked!”

D) Something else?

Extension block limitations

- Cannot contain nested types
- Do not introduce a new “scope”
- Do not have a name – cannot be referenced
- Cannot add state: no fields, no events, no auto nor field properties
- No custom conversion operators
- No constructors – use static factory methods instead

Use Case: Domain-specific additions

```
public static class HttpStatusCodeExtensions
{
    extension(HttpStatusCode status)
    {
        public bool IsSuccess => (int)status >= 200 && (int)status < 300;
        public bool IsRedirect => (int)status >= 300 && (int)status < 400;
        public bool IsClientError => (int)status >= 400 && (int)status < 500;
        public bool IsServerError => (int)status >= 500 && (int)status < 600;
    }
}

var response = await httpClient.GetAsync(url);
if (response.StatusCode.IsSuccess)
{
    Console.WriteLine("Success!");
}
else if (response.StatusCode.IsClientError)
{
    Console.WriteLine("Client error - check your request");
}
```

Special Thanks to Dave Brock

https://www.daveabrock.com/2025/12/05/extension-properties-c-14s-game-changing-feature-for-cleaner-code/

Use Case: Domain-specific additions

```
public static class StringExtensions
{
    extension(string str)
    {
        public bool IsEmpty => string.IsNullOrEmpty(str);
        public bool IsWhitespace => string.IsNullOrWhiteSpace(str);
        public bool IsValidEmail => !str.IsEmpty
                                    && str.Contains("@") && str.Contains(".");
        public bool IsNumeric => !str.IsEmpty && str.All(char.IsDigit);
    }
}

string email = "sara@peloton.com";
if (!email.IsEmpty && email.IsValidEmail)
{
    Console.WriteLine("Valid email!");
}
```

Special Thanks to Dave Brock

https://www.daveabrock.com/2025/12/05/extension-properties-c-14s-game-changing-feature-for-cleaner-code/

Use Case: Polyfills
bring *new* C#/.NET to *old* .NET

- Newer versions of .NET add lots of new types and members
- Sometimes extend existing types:
  - `System.Random.Shared`
  - `<ExceptionType>.ThrowIf<Condition>` methods
- Many C# features depend on the presence of well-known attributes and types
  - `System.Index`, `System.Range`
  - Nullable Reference Types

Special Thanks to Gérald Barré aka meziantou
https://www.meziantou.net/csharp-14-extension-members-enhancing-polyfill-libraries.htm

Use Case: Polyfills
How it works under the hood

- Most of this is already possible with extension methods
  - Properties and static methods add a new angle
- Extensions do not cause conflicts
- Existing members of the extended types always considered first
- No errors, no warnings

Special Thanks to Gérald Barré aka meziantou
https://www.meziantou.net/csharp-14-extension-members-enhancing-polyfill-libraries.htm

Use Case: “Enums on steroids” via Source Generators

- Adds members to your enum types – making them feel natural
- Allows to bypass slow reflection by generating fast alternatives

```
[EnumExtensions]
public enum MyColours
{
    Red,
    Green,
    Blue,
}
```

Special Thanks to Andrew Lock
https://andrewlock.net/exploring-dotnet-10-preview-features-3-csharp-14-extensions-members/#a-case-study-netescapades-enumgenerators

Assemblies

- ExtensionsDemo (1.0.0.0, .NETCoreApp, v10.0)
  - Metadata
  - Debug Metadata (From portable PDB)
  - References
  - { } -
    - <Module> @02000001
    - Extensions @02000003
      - Base Types
      - extension<T>(IEnumerable<T> source)
        - Empty : IEnumerable<T> @17000002
        - IsEmpty : bool @17000001
        - FirstOrDefault(T) : T @0600000c
        - operator +(IEnumerable<T>, IEnumerable<T>) : IEnumerable<T> @0600000b
        - PrintAll() : void @0600000d
      - Test() : void @06000008
    - Program @02000002
- System.Runtime (10.0.0.0, .NETCoreApp, v10.0)
- System.Collections (10.0.0.0, .NETCoreApp, v10.0)
- System.Console (10.0.0.0, .NETCoreApp, v10.0)
- System.Linq (10.0.0.0, .NETCoreApp, v10.0)
- System.Private.CoreLib (10.0.0.0, .NETCoreApp, v10.0)
- System.Private.Uri (10.0.0.0, .NETCoreApp, v10.0)
- System.Runtime.InteropServices (10.0.0.0, .NETCoreApp, v10.0)
- System.Runtime.CompilerServices.Unsafe (10.0.0.0, .NETCoreApp, v10.0)

Extensions @02000003

```
1  // ExtensionsDemo, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
2  // Extensions
3  using ...
6
7  public static class Extensions
8  {
9      extension<T>(IEnumerable<T> source)
10     {
11         public bool IsEmpty => !source.Any();
12
13         public static IEnumerable<T> Empty => Enumerable.Empty<T>();
14
15         public static IEnumerable<T> operator +(IEnumerable<T> left, IEnumerable<T> right)
16         {
17             return left.Concat(right);
18         }
19
20         public T FirstOrDefault(T defaultValue)
21         {
22             source.PrintAll();
23             T val = source.FirstOrDefault();
24             return (val != null) ? val : defaultValue;
25         }
26
27         private void PrintAll()
28         {
29             foreach (T item in source)
30             {
31                 Console.WriteLine(item);
32             }
33         }
34     }
35
36     public static void Test()
37     {
38         List<int> list1 = new List<int> { 1, 2, 3 };
39         List<int> list2 = new List<int> { 4, 5, 6 };
40         IEnumerable<int> combined = list1 + list2;
41         Console.WriteLine(get_IsEmpty(combined));
42         Console.WriteLine(combined.FirstOrDefault(0));
43         IEnumerable<int> emptyList = get_Empty<int>();
44         Console.WriteLine(get_IsEmpty(emptyList));
45     }
46 }
47
```

Stumbling Blocks

- We were too early (again)… “encoding” changed from one preview to the next
- Type or no type – that’s the question!
- Generic parameters on extension blocks are weird
- Some extra work needed to make extension members fit into the ILSpy UI

Roslyn Internals

- Looked at public API for extension blocks in Roslyn
- Not much to see ☹
- Extension blocks are just nested types

[0] {Extensions.extension<T>(IEnumerable<T>)}

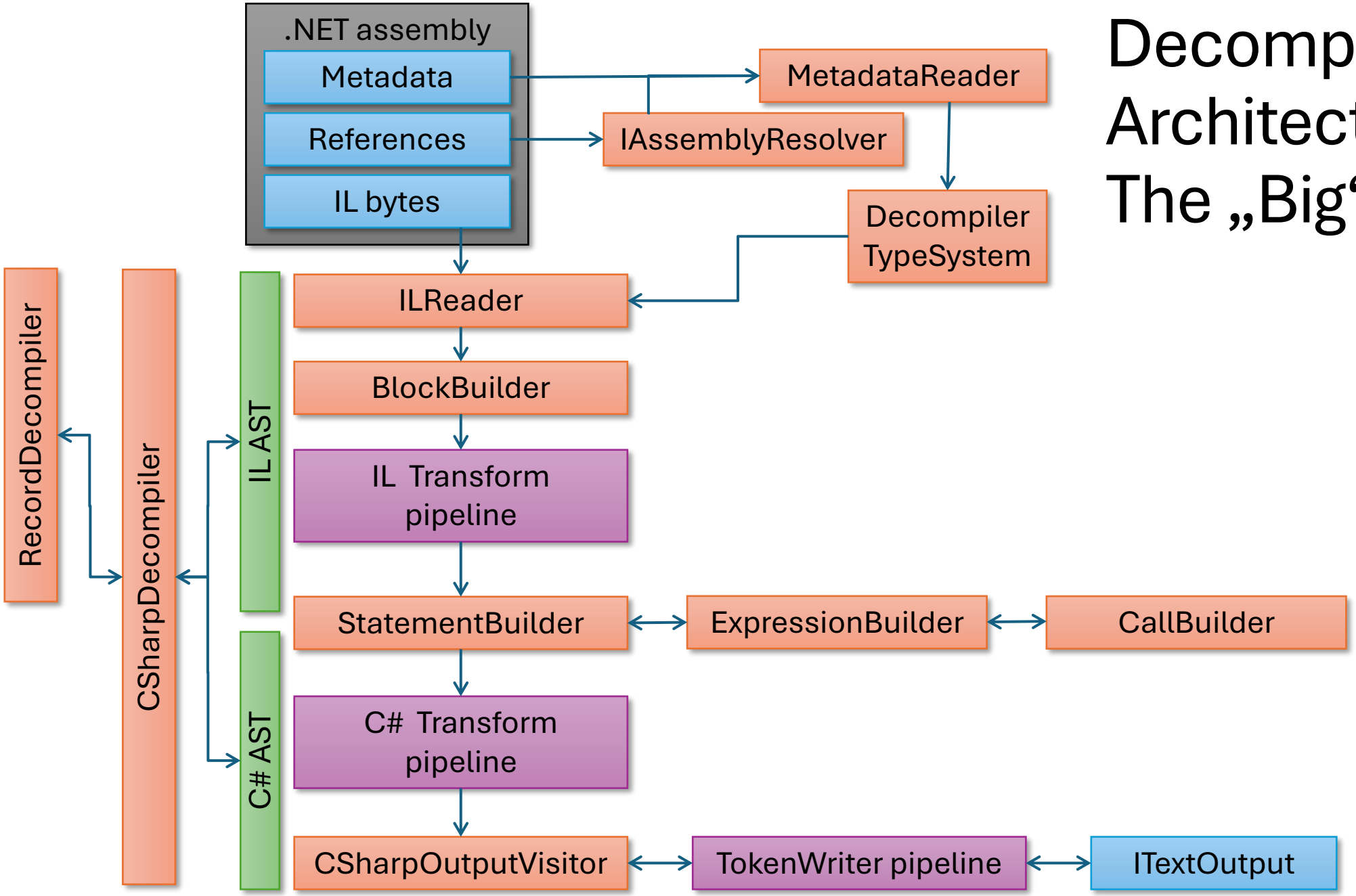
Microsoft.CodeAnalysis.INamedTypeSymbol.Arity		1
Microsoft.CodeAnalysis.INamedTypeSymbol.AssociatedSymbol		null
Microsoft.CodeAnalysis.INamedTypeSymbol.ConstructedFrom		{Extensions.extension<T>(IEnumerable<T>)}
Microsoft.CodeAnalysis.INamedTypeSymbol.Constructors	View	Length = 0
Microsoft.CodeAnalysis.INamedTypeSymbol.DelegateInvokeMethod		null
Microsoft.CodeAnalysis.INamedTypeSymbol.EnumUnderlyingType		null
Microsoft.CodeAnalysis.INamedTypeSymbol.ExtensionGroupingName	View	"<G>$1AC5E520DA86B82B354ED395EFEDAFEC"
Microsoft.CodeAnalysis.INamedTypeSymbol.ExtensionMarkerName	View	"<M>$68D01D9B725350F72AB3FAA90B677685"
Microsoft.CodeAnalysis.INamedTypeSymbol.ExtensionParameter		{IEnumerable<T> source}
Microsoft.CodeAnalysis.INamedTypeSymbol.InstanceConstructors	View	Length = 0
Microsoft.CodeAnalysis.INamedTypeSymbol.IsComImport		false
Microsoft.CodeAnalysis.INamedTypeSymbol.IsExtension		true
Microsoft.CodeAnalysis.INamedTypeSymbol.IsFileLocal		false
Microsoft.CodeAnalysis.INamedTypeSymbol.IsGenericType		true
Microsoft.CodeAnalysis.INamedTypeSymbol.IsImplicitClass		false

Down the Rabbit Hole

Decompiler Architecture

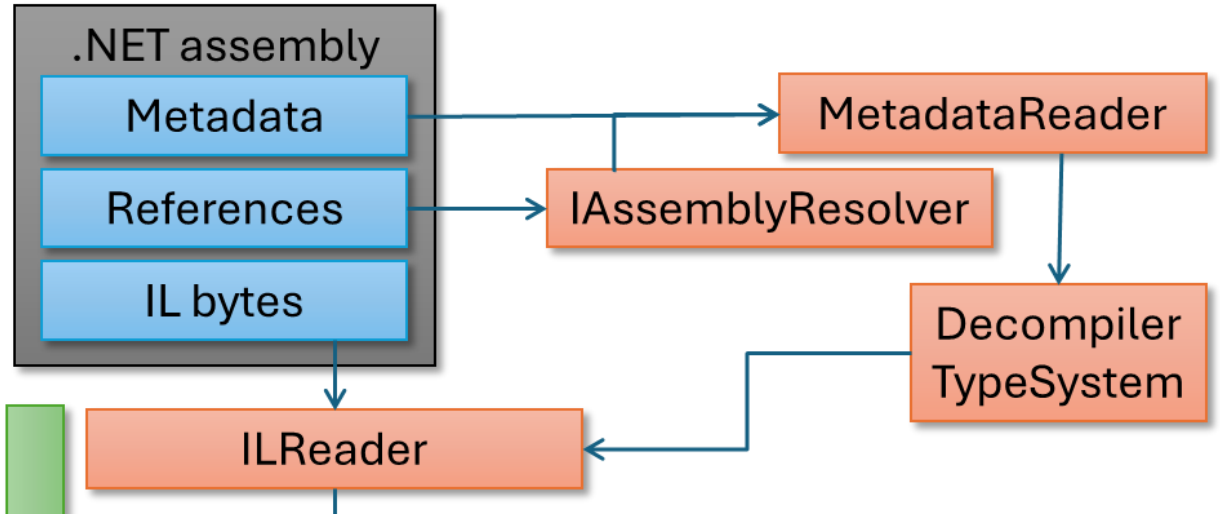
Finally... ☺

Decompiler Architecture: The „Big“ Picture



The Input

- Metadata from input assembly
- Metadata from *transitively* referenced assemblies
- IL bytes from input assembly
- References may be “reference assemblies”



Metadata

- IL without metadata == useless
- Stores info about types, members, references, etc.
- Managed assemblies are relational “databases”
  - Each table has a number (e.g. 02 = TypeDef)
  - Each table has a primary key (the row number)
  - A row may be referenced in another table via a token (table number + row number)
    `02000001` = first type definition in the module

Want to know more? Watch this video by David Wengier
https://www.youtube.com/watch?v=Buxmr0juFoo

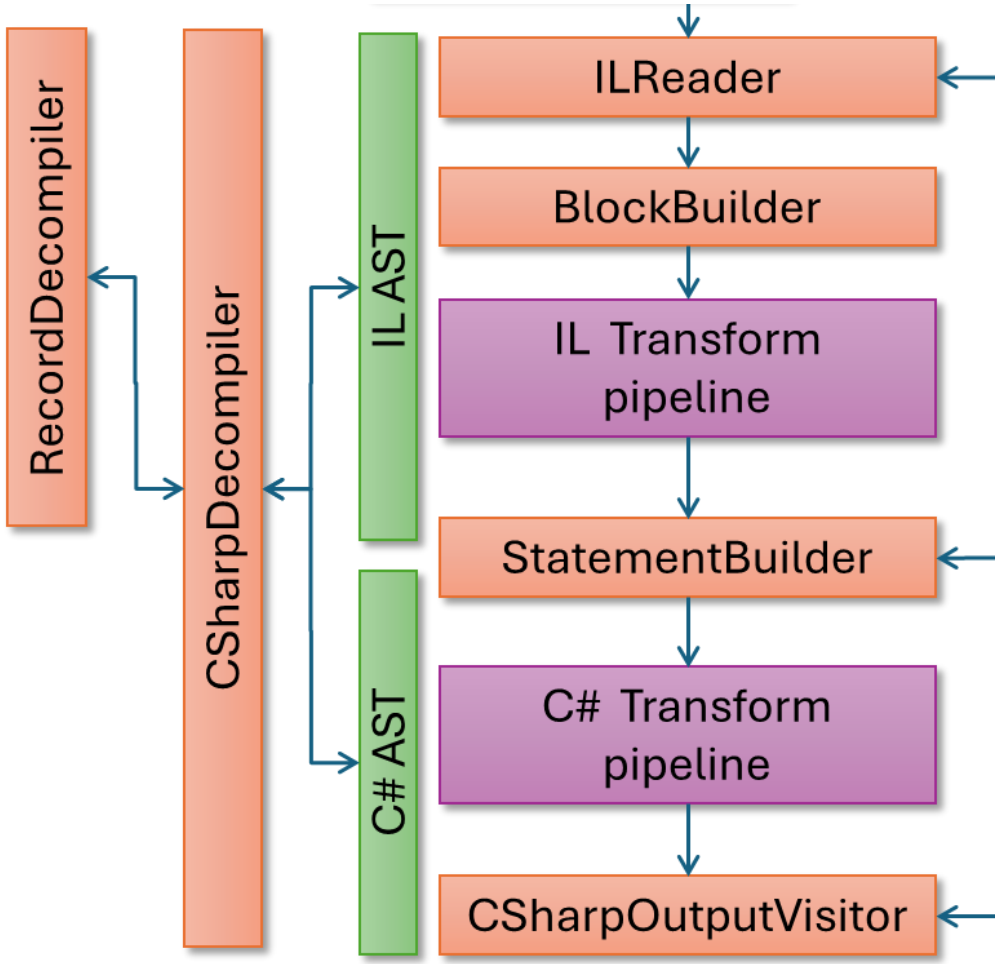
Metadata

- System.Reflection.Metadata
  - Metadata tables directly accessible
  - Pro: fast and lean
  - Con: No convenient API, low-level knowledge required

- Decompiler.TypeSystem
  - High-level representation of metadata
  - Language-specific (C#)
    - Member Lookup
    - Type Conversions
    - Overload Resolution
  - Comparable to Roslyn

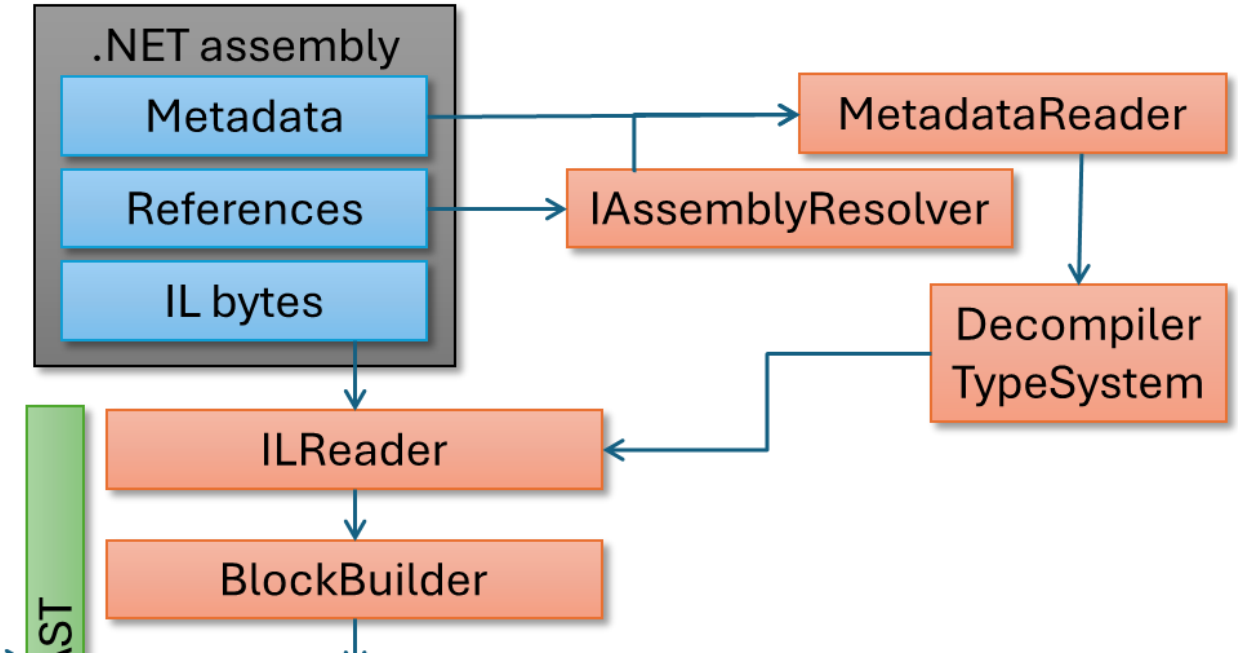
CSharpDecompiler: The driver/entry point

- Inputs:
  - Decompiler type system
  - Options
- Offers APIs to decompile members, types, etc.
- Decides which types and members are decompiled
  - In records most members are compiler-generated/hidden
  - Extension blocks are created here
- Loads method bodies via ILReader
- Drives transformation pipeline
- Returns either the C# AST or a string



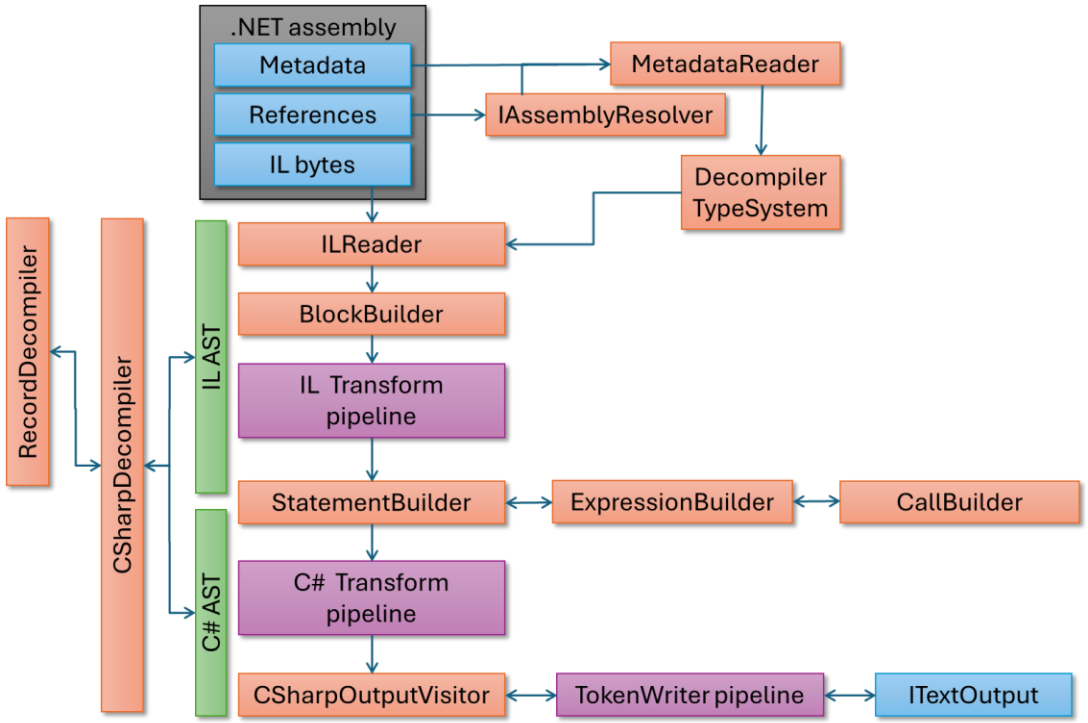
Pipeline Step #1: ILReader + BlockBuilder

- Decodes IL instruction stream
- Normalizes instructions
- Transform stack-based IL assembly into statement-based ILAst
- Infer types of “stack-slots”
- Detect basic blocks by analyzing branch instructions
- Make implicit conversions explicit where possible



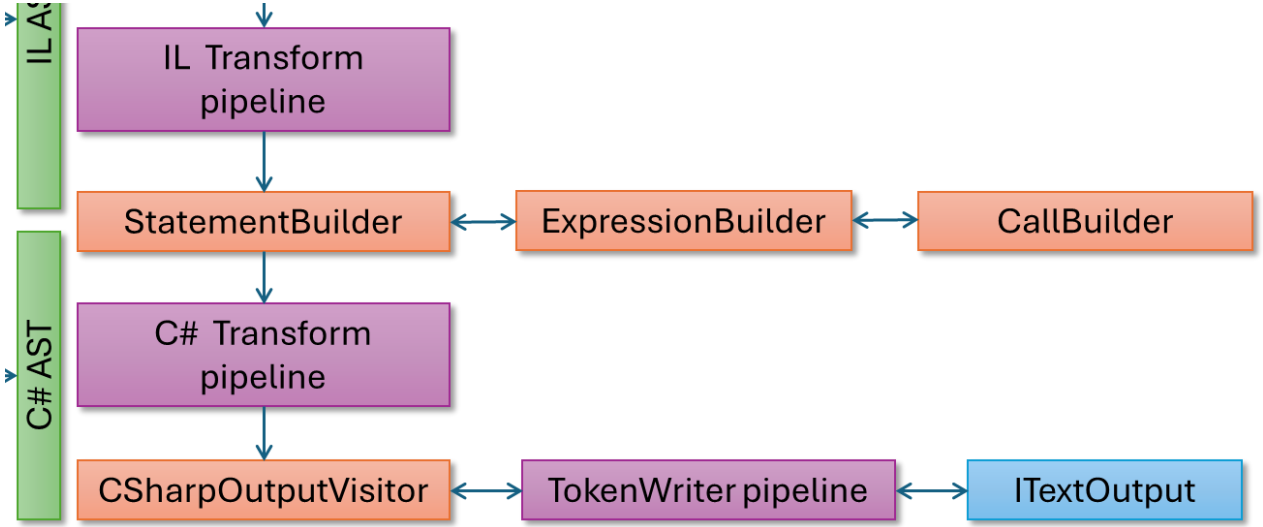
Pipeline Step #2: ILAst / ILAst Transforms

- Annotated and structured representation of IL instructions
  - Blocks: try, catch, loops, using, lock, switch
  - Statements: leave, branch
- Every instruction has an explicit result type
- ILAst Transforms modify the structure, but not the semantics of the code, e.g.
  - Transform conditional branches into IfInstructions
  - Inline blocks



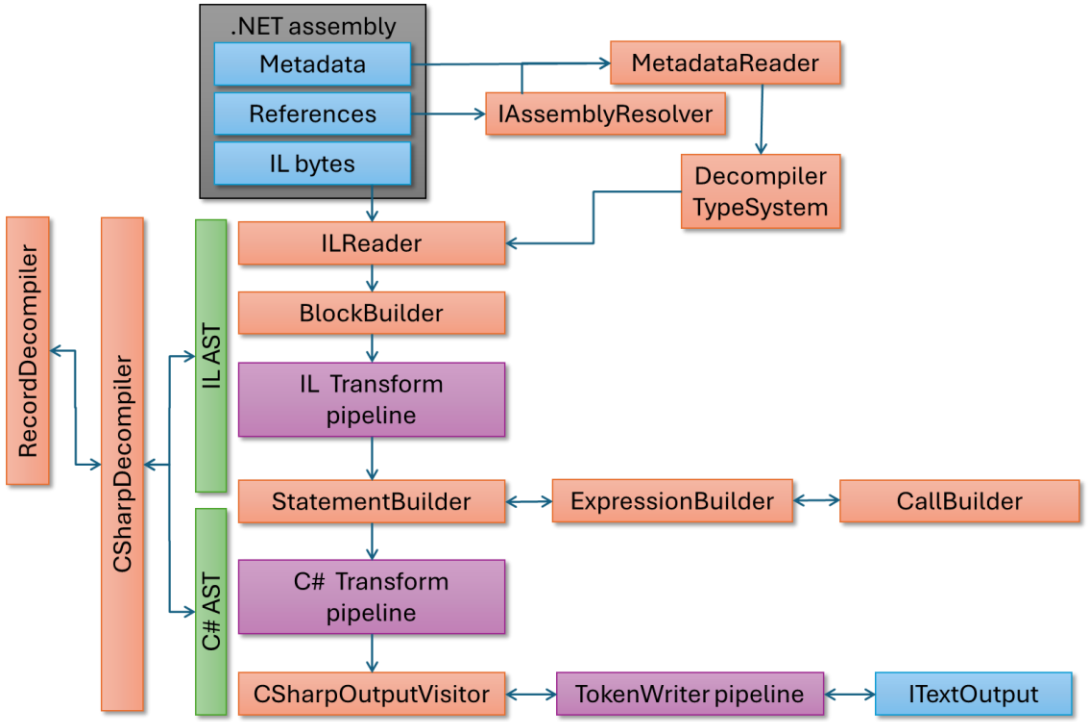
Pipeline Step #3: Build C# code

- StatementBuilder
  - Generate C# statements
- ExpressionBuilder
  - Translate to C# expressions
  - Convert Stack type to C# type
  - Eliminate conversions where possible
- CallBuilder
  - Translate calls to property access, method calls
  - Ensure correct method is called → overload resolution



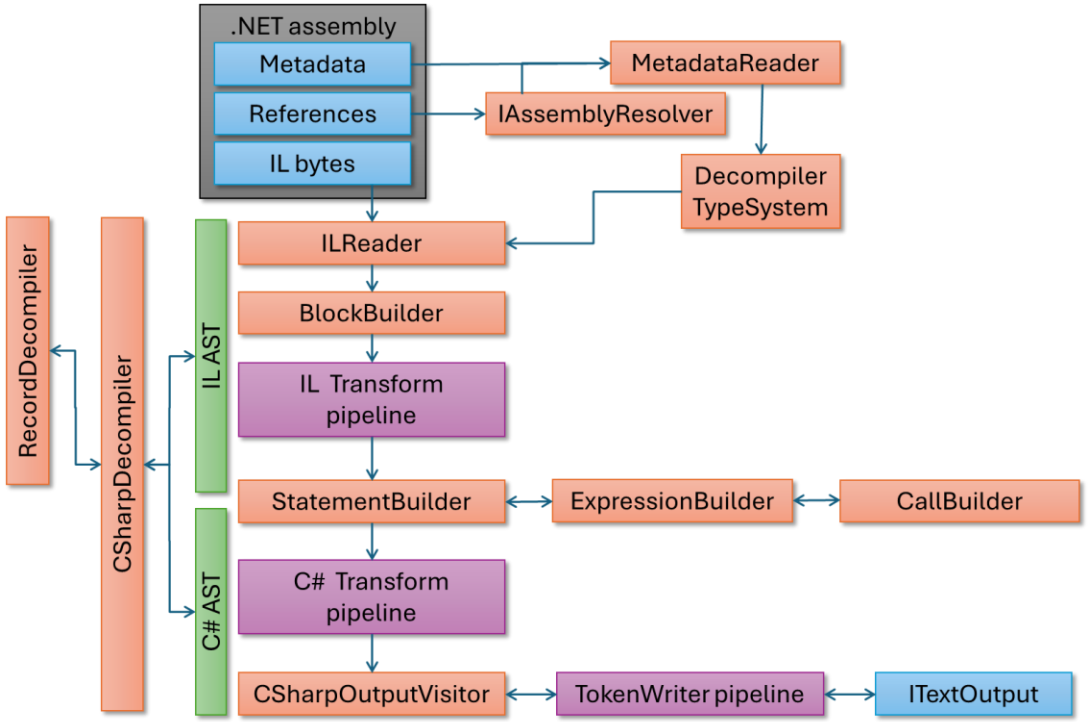
Pipeline Step #4: C# Transforms

- “Beautify” the generated code
- Replace method calls with operators
- Declare Variables
- Introduce using declarations
- Introduce extension method syntax
- Transform extension method chains to LINQ expression syntax



Pipeline Step #5: Produce Output

- CSharpOutputVisitor turns AST into tokens
- TokenWriter pipeline performs translation into characters
  - Inserts comments/directives
  - Extensible to allow handling links, tooltips and semantic highlighting



Extensions in ILSpy

What already works

- Reconstruct extension blocks from compiler-generated code
  - Not entirely loss-less
- Extension operators call-sites (little happy accident)

What‘s missing

- Use “correct” declaring class for static extension members
- Turn get/set method calls into property accesses
- Not strictly necessary, static method call syntax is perfectly fine

829 + Q&A